

TreeHeap：面向端侧 AI 的多分辨率递归状态架构

定位：基于可复现实验结果提出的工程研究方向，不作为量产性能承诺。

1. 技术判断

现有序列模型通常在统一分辨率上维护和读取状态。上下文增长后，即使当前任务只需要粗粒度判断，系统仍可能承担细粒度状态保存、搬运和计算的成本。TreeHeap 尝试改变这一点：它把序列状态组织成一棵具有 parent、detail 和 address 的递归树，使同一段输入能够同时保留粗粒度摘要和细粒度残差，并允许 READ 按任务需要选择读取范围。

这不是对 Transformer 的局部算子替换，而是对“模型状态如何组织、保存和读取”的重新设计。其工程目标是：在质量约束下，让计算和内存访问随所需分辨率变化，而不必始终以完整细粒度状态工作。

TreeHeap 已完成基础架构设计和多项结构验证。下一阶段将面向移动设备，重点验证多分辨率递归状态对峰值内存、内存带宽和每 token 能耗的实际影响，并据此判断产品化价值。

2. 架构定义

设叶节点状态为 h_i 。FOLD 对相邻状态进行递归组合，同时保留可用于恢复的 detail：

$$(p_i, d_i) = \text{FOLD}(h_{2i}, h_{2i+1}; \theta_f)$$

其中 p_i 是父节点状态， d_i 是局部残差信息， θ_f 为共享或分层参数。递归执行后得到从 leaf 到 root 的多分辨率状态集合：

$$H = \{H_{\text{leaf}}, H_{\text{parent1}}, \dots, H_{\text{root}}, D\}$$

UNFOLD 使用 parent、detail 和 address 恢复下一层状态：

$$(h_{\hat{2}i}, h_{\hat{2}i+1}) = \text{UNFOLD}(p_i, d_i, \text{address}_i; \theta_u)$$

READ 不等同于完整逆变换。它接收树状态和读取预算，形成供 decoder 使用的任务状态：

$$z = \text{READ}(H, \text{budget}, \text{query}; \theta_r)$$

这里有三个必须区分的工程对象：

- parent：两个或多个子状态经过 FOLD 后形成的组合状态；
- address：节点在树中的静态坐标，例如深度和左右分支序列；
- recursive path：从 leaf 到目标节点实际经过的算子序列和中间状态，是计算历史，不只是坐标标签。

因此，TreeHeap 的核心并不是“把 token 放进树里”，而是建立一套可训练的多分辨率状态协议：FOLD 决定如何形成层级状态，UNFOLD 检查信息闭合，READ 决定哪些分辨率进入生成过程。

3. 为什么值得研究

3.1 状态分辨率成为显式控制量

在普通序列表示中，粗粒度语义通常隐含在高维状态中，系统难以直接指定“只读取概览”或“补充局部细节”。TreeHeap 把层级状态暴露为结构对象，使 coarse-only、coarse-plus-detail 和 full-detail 成为可执行的读取策略。若该机制在真实任务上成立，端侧系统就可能按质量和资源预算动态选择计算路径。

3.2 信息闭合与任务读出被分开验证

可逆或近似可逆的 FOLD/UNFOLD 用于回答“状态是否在递归中丢失”；READ/decoder 用于回答“保存的信息能否被任务利用”。这两项性质不能互相替代。分开测量，可以避免把 echo、哈希记忆或 teacher-forced NLL 误写成语言能力。

3.3 资源收益存在明确的归因路径

TreeHeap 的端侧价值不依赖“模型更神奇”的叙述，而依赖可测的工程链条：读取更少的 detail，减少活跃状态；活跃状态减少，降低内存访问和算子调用；在质量门槛内比较延迟、带宽与能耗。若链条中任一环节不成立，该方向即可被否定或收缩。

4. 当前证据

4.1 固定容量结构能够参与学习

固定容量、地址敏感和子堆结构实验显示：仅使用 flat address 或 path-only 表示时，结构定位任务不能完成；引入 subheap/path 结构后，任务准确率由 0.0 提升到 1.0。

工程含义：结构关系不是日志中的附加标签，而可以进入模型状态并影响可学习行为。该结果证明结构机制可用，但不单独证明语言生成能力。

证据：ara/m0-treeheap-math/evidence/structural_c05_probe/

4.2 多分辨率状态可以渐进恢复

在冻结 root compressor、为内部地址配置有界 detail code 的 codec 实验中，随着 detail 逐步加入：

- 重建 MSE: 1.9334 -> 0.5526
- k=64 token top-1: 0.9964
- sequence exact: 0.8268
- 地址错位 MSE: 3.3147
- root NLL 增量: 0

工程含义：root 与 detail 构成了可测的渐进编码；错误地址会显著破坏恢复，说明 detail 不是可任意交换的旁路缓存。该证据支持 codec 层面的多分辨率恢复，不等价于高层语言 READ 已经成立。

证据：ara/s3-generation/evidence/s3_multiresolution_treeheap_pyramid/main_v2/

4.3 递归变换可以保持数值闭合

在 lifting pump 实验中：

- depth-6 closure 最大误差: 3.70e-6
- state MSE: 3.14e-14
- 原生 token/block echo: 1.0 / 1.0
- root 替换导致 next-token NLL: +0.21359

工程含义：多层递归本身不必导致数值信息不可控地消失；root 对后续读出具有部分因果作用。原注册的 root-echo 强 Claim 因 token drop 未达到 0.10

而失败，因此目前只能确认数值闭合和部分读出作用，不能宣称已经形成语义层级。

证据：ara/s1-echo/evidence/s1_lifting_information_pump/main/

4.4 parent 与 address 对读出具有可干预性

内部 echo 实验中，破坏最近 parent 或 address 后，NLL 分别变化：

- closest-parent: +74.4254
- address: +54.3687

- multichannel top-1/exact: 0.999428 / 0.991211

工程含义：共享 FOLD 能把局部 token 对写入 parent 状态，decoder 的正确读出依赖 parent 和 address。与此同时，READ 的 99.9786% 权重集中在第一层，说明当前语言任务仍主要使用 near-leaf 状态；“高层多分辨率语言读出”仍是开放问题，而不是既成结论。

证据：ara/s1-echo/evidence/s1_multiresolution_internal_echo/main/

5. 已建立的边界

技术命题	证据状态	工程判断
固定容量递归状态可训练	已支持	可以继续作为架构底座
FOLD/UNFOLD 数值闭合	已支持	递归深度不必天然造成信息丢失
codec 多分辨率渐进恢复	已支持	root/detail/address 机制成立
parent/address 影响读出	已支持	结构变量具有因果可干预性
独立 recursive-path 因果性	部分支持	需与静态 address 解耦验证
高层状态承载语言级 coarse 语义	尚未证明	READ 仍偏向 near-leaf
真实移动设备降低内存和功耗	尚未测试	必须通过部署基准验证
在通用质量上超过 Transformer	无证据支持	不作为当前立项目标

这些边界不是削弱方案，而是给出了清晰的研究切面：数值闭合和 codec 已经越过可行性门槛；真正需要攻克的是语言状态的跨层形成、READ 的分辨率选择，以及结构收益在硬件上的兑现。

6. 下一阶段工程验证

6.1 模型合同

- 使用相同语料、tokenizer、seed、训练步数和有效 token 数；
- 对比 TreeHeap、flat sequence 和参数量匹配的 Transformer；
- 单独记录 FOLD、READ、decoder 的参数量、激活量和运行时间；
- 保留相同初始化或 step-0 函数一致性检查，避免把初始化差异误判为架构收益。

6.2 读取预算

对同一 checkpoint 测试三种状态预算：

- coarse-only：仅使用 root 或高层 parent；
- coarse-plus-detail：按固定预算补充 detail；
- full-detail：读取完整层级状态，作为质量上限参考。

重点观察质量是否随读取预算平滑变化。如果质量只能在 full-detail 下成立，TreeHeap 就没有形成可利用的多分辨率协议；如果在部分 detail 下保持质量，则可继续测量资源收益。

6.3 端侧指标

- 模型文件与常驻内存；
- 峰值激活内存和 KV/状态缓存；

- DRAM 读写量与算子调用次数；
- 首 token 延迟、稳态 tokens/s；
- 平均功率、峰值功率和每 token 能耗；
- 输出非空率、重复率、NLL/PPL，以及任务级质量指标。

测量应在至少一个真实端侧后端完成。CPU 仿真只能用于算子正确性和趋势筛选，不能代替移动 NPU/GPU 的功耗结论。

6.4 判定方式

本阶段不设置基于单次 NLL

波动的提前停止门。完整训练预算跑完后，在预先确定的质量容差内比较资源指标。只有当收益能够由读取层级或 detail 预算稳定控制，并且相对匹配基线仍然存在，才能归因于 TreeHeap 的多分辨率结构。

7. 预期交付

本课题将交付：

- 可复现的 TreeHeap 训练与推理原型；
- FOLD、UNFOLD、READ 和 decoder 的模块化实现及消融接口；
- 面向 coarse/detail 读取预算的统一评测工具；
- 与匹配 Transformer 基线的端侧资源报告；
- 包含失败实验、边界条件和 checkpoint 哈希的证据包。

项目的阶段性成功标准不是立即替代主流架构，而是回答一个具有产品价值的问题：在可接受的生成质量下，递归多分辨率状态能否让端侧模型少保存、少搬运、少计算。如果答案为是，TreeHeap 将提供一条不同于单纯量化、剪枝和缩小模型规模的架构路径；如果答案为否，完整证据链也能明确指出限制来自状态形成、读取协议还是硬件映射。

8. 立项建议

建议将 TreeHeap 作为“面向移动设备低功耗 AI 的新型状态架构”开展小规模工程预研。现阶段已有结果足以支持进入真实任务和真实硬件验证，但不足以承诺功耗优势或量产质量。

这项工作的创新点不在于预先宣称一个更强模型，而在于把状态分辨率、递归闭合和按需读取变成可训练、可干预、可部署测量的架构对象。它提供了一个明确的新问题，也提供了可以用工程数据回答这个问题的方法。